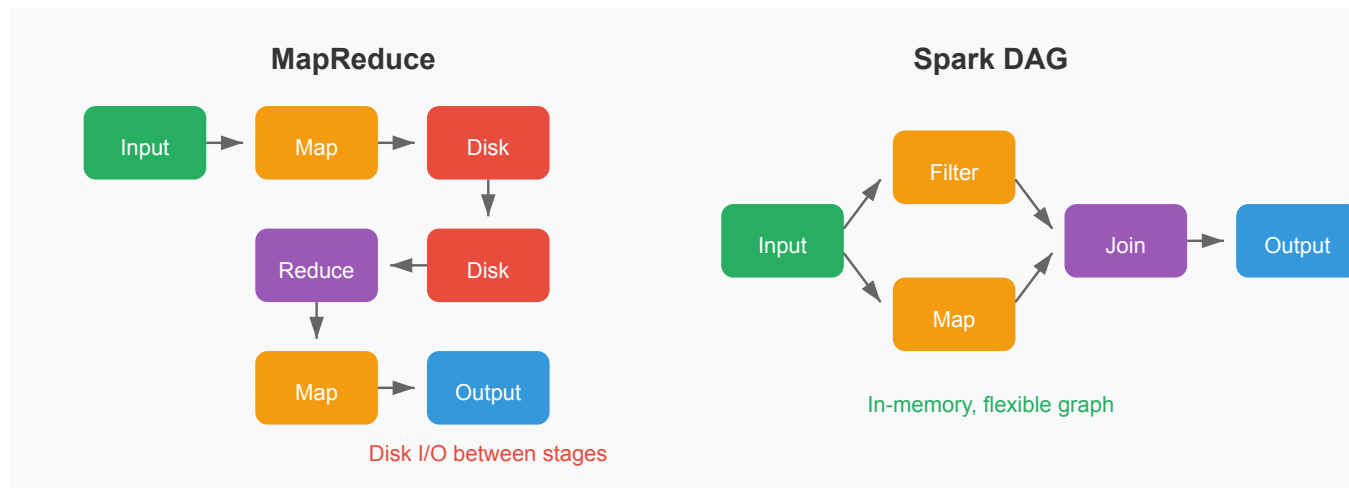


Modern Batch Processing Beyond MapReduce

 systemdesignschool.io/fundamentals/modern-batch-spark



Modern Batch: Spark

MapReduce established distributed batch processing, but working with it felt like writing assembly code. Every computation must fit the rigid map-shuffle-reduce pattern. Intermediate results write to disk between jobs. Iterative algorithms crawl.

Apache Spark addressed these limitations while preserving MapReduce's core insights about distributed data and fault tolerance. Today, Spark handles most batch workloads that once ran on MapReduce.

Why MapReduce Became Limiting

Consider what MapReduce requires for a 10-stage data pipeline. Each stage is a separate job. Between jobs, you write intermediate results to HDFS and read them back. That's 10 writes and 10 reads to distributed storage, even if the entire dataset fits in the cluster's aggregate memory.

Rigid structure. Every MapReduce job follows the same pattern: map phase, shuffle, reduce phase. Some algorithms need different access patterns. A graph algorithm that iteratively updates node values requires a full MapReduce job per iteration—the framework can't express "iterate until converged" as a single job.

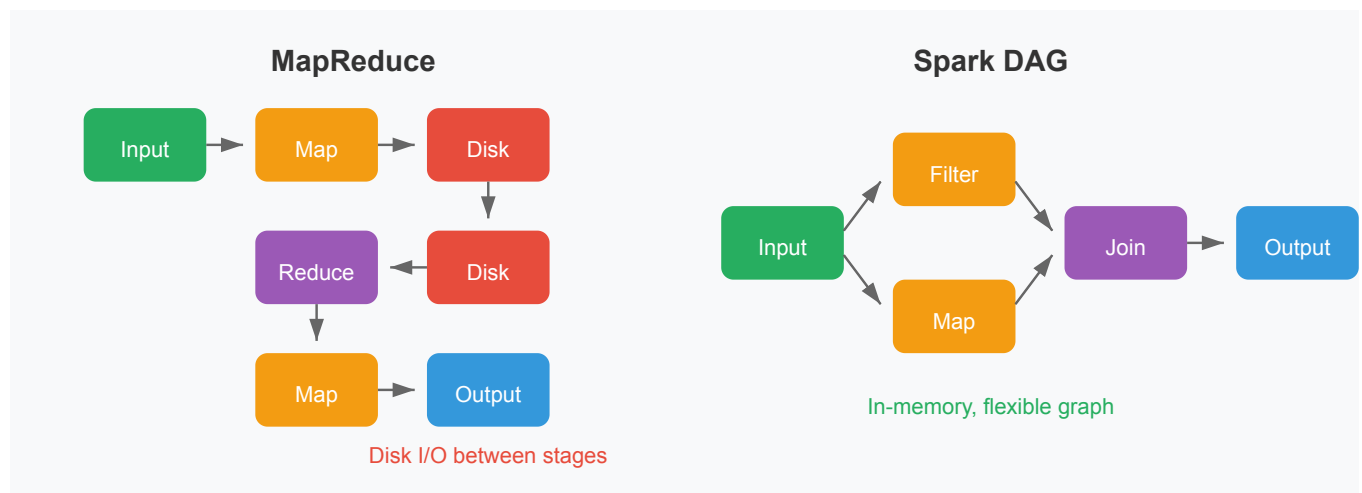
Disk overhead between stages. MapReduce writes map outputs to local disk, shuffles them, then writes reduce outputs to HDFS. A multi-stage pipeline pays this disk I/O tax repeatedly. For small intermediate datasets that fit in memory, this overhead dominates runtime.

Poor iteration support. Machine learning algorithms iterate until convergence. Gradient descent might need hundreds of passes over the training data. Each MapReduce iteration reads from HDFS, computes gradients, writes updated parameters to HDFS. The disk overhead per iteration makes iterative ML impractical.

How Spark Changed the Model

Spark introduced three key improvements that address MapReduce's limitations.

Flexible DAG execution. Instead of forcing everything into map-shuffle-reduce, Spark represents computations as a directed acyclic graph (DAG) of operations. You compose transformations (filter, map, join, aggregate) and Spark's optimizer arranges them efficiently—combining stages where possible, pushing filters early, choosing join strategies.



In-memory caching. Spark can keep intermediate data in memory across stages. A 10-stage pipeline reads input once, processes through all stages in memory, and writes output once. When data doesn't fit in memory, Spark spills to disk—graceful degradation rather than failure.

Richer operation vocabulary. Beyond map and reduce, Spark provides joins, aggregations, window functions, and SQL queries out of the box. The same engine handles batch ETL, interactive SQL queries, machine learning, and graph processing.

Concrete Comparison: Log Analysis

Let's trace the same log analysis job through both frameworks.

MapReduce approach:

1. Job 1: Read logs from HDFS → Map (extract fields) → Shuffle → Reduce (count by URL) → Write to HDFS
2. Job 2: Read counts from HDFS → Map (identity) → Shuffle (sort by count) → Reduce (top-N) → Write to HDFS

Each job involves disk writes for intermediate data. Two jobs means four HDFS operations plus local disk for shuffle.

Spark approach:

1. Read logs from HDFS
2. Transform in memory: extract fields → group by URL → count → sort → take top N
3. Write results to HDFS

Spark reads once, transforms through a single DAG (potentially with one shuffle for the groupBy), and writes once. For datasets that fit in memory, the difference is substantial.

When the Difference Is Dramatic: Iteration

The gap widens for iterative algorithms. Consider PageRank, which iteratively updates page scores based on link structure.

MapReduce: Each iteration is a separate job. Read graph + current scores from HDFS → compute new scores → write to HDFS. Repeat until convergence. If PageRank needs 20 iterations, that's 40 HDFS read/write operations.

Spark: Load graph into memory once, cache it. Each iteration updates scores in memory. Write final results to storage. Total: 1 read + 1 write, regardless of iteration count.

This is why Spark's original paper emphasized iterative machine learning—the performance difference can be 10-100x for algorithms that iterate over cached data.

When to Use Spark

Spark handles most workloads that historically ran on MapReduce:

- **ETL pipelines** with multiple transformation stages benefit from DAG optimization and reduced disk I/O
- **Machine learning** on historical data leverages in-memory iteration
- **Interactive queries** get faster turnaround than batch-scheduled MapReduce jobs
- **SQL analytics** through Spark SQL provides a familiar interface with distributed execution

Spark still reads from and writes to distributed storage (HDFS, S3, cloud object storage). The improvement is in how computation executes between storage operations—not eliminating storage, but reducing redundant trips.

Limitations and Trade-offs

Spark isn't free of trade-offs. Memory caching requires sufficient cluster memory; datasets larger than aggregate memory spill to disk, reducing the advantage. Spark's query optimizer occasionally makes poor choices that require manual tuning. And for simple one-pass transformations, the difference versus MapReduce may be negligible.

What's Next

So far we've covered batch processing: scheduled jobs that process bounded datasets. But some applications can't wait hours for results. Fraud detection needs to flag suspicious transactions in seconds. Dashboards need live updates. Sensors generate continuous data streams.

The next section introduces stream processing—handling unbounded data continuously as it arrives rather than in scheduled batches.